

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
КАФЕДРА АВТОМАТИЗАЦІЇ ТА ІНФОРМАЦІЙНИХ СИСТЕМ

ЗВІТ

ЗВІТ З ЛАБОРАТОРНИХ РОБІТ
З НАВЧАЛЬНОЇ ДИСЦИПЛІНИ
«ТЕОРІЯ ПРИЙНЯТТЯ РІШЕНЬ»

Виконав студент групи КН-23-1

Полинько Ігор Миколайович

Перевірів: старший викладач Рилова Н. В.

Кременчук 2025

ЛАБОРАТОРНА РОБОТА № 7

Тема: Розв'язання задачі теорії ігор. Матричні ігри

Мета: опанування методів спрощення й розв'язання матричних ігор.

Завдання на лабораторну роботу:

Дана платіжна матриця розміром $m \times n$.

1. Спростити матрицю, якщо це можливо.
2. Знайти сідлову точку, якщо вона є.
3. Якщо сідлової точки немає, поставити та розв'язати задачу знаходження змішаних стратегій.

Додаток Г:

Варіант № 15.

-1	-4	5	3
7	7	4	-2
5	4	-3	6
0	-4	6	5

Варіант: 15

Код:

```
import numpy as np
from scipy.optimize import linprog

# 1. Початкова платіжна матриця (варіант 15)
A = np.array([
    [-1, -4, 5, 3],
    [ 7,  7, 4, -2],
    [ 5,  4, -3, 6],
    [ 0, -4, 6, 5]
], dtype=float)

print("Початкова матриця:")
print(A)

# --- 1. Спрощення матриці (видалення домінованих рядків та стовпців) ---
def remove_dominated_rows(matrix):
    rows_to_keep = []
    for i in range(len(matrix)):
        dominated = False
        for j in range(len(matrix)):
            if i != j and all(matrix[j] >= matrix[i]) and any(matrix[j] >
matrix[i]):
                dominated = True
                break
        if not dominated:
            rows_to_keep.append(i)
    return matrix[rows_to_keep]
```

```

        rows_to_keep.append(i)
    return matrix[rows_to_keep, :]

def remove_dominated_cols(matrix):
    matrix_t = matrix.T
    cols_to_keep = []
    for i in range(len(matrix_t)):
        dominated = False
        for j in range(len(matrix_t)):
            if i != j and all(matrix_t[j] <= matrix_t[i]) and any(matrix_t[j]
< matrix_t[i]):
                dominated = True
                break
        if not dominated:
            cols_to_keep.append(i)
    return matrix[:, cols_to_keep]

A_simple = remove_dominated_rows(A)
A_simple = remove_dominated_cols(A_simple)
print("\nСпрощена матриця:")
print(A_simple)

# --- 2. Знаходження сідлової точки ---
row_min = A_simple.min(axis=1)
col_max = A_simple.max(axis=0)
maximin = row_min.max()
minimax = col_max.min()

row_index = np.where(row_min == maximin)[0]
col_index = np.where(col_max == minimax)[0]

saddle_points = [(i,j) for i in row_index for j in col_index if A_simple[i,j]
== maximin]

if saddle_points:
    print("\nСідлова точка:", saddle_points)
    print("Ціна гри:", maximin)

```

Результат:

```

Початкова матриця:
[[-1. -4.  5.  3.]
 [ 7.  7.  4. -2.]
 [ 5.  4. -3.  6.]
 [ 0. -4.  6.  5.]]

Спрощена матриця:
[[ 7.  4. -2.]
 [ 4. -3.  6.]
 [-4.  6.  5.]]

Сідлова точка є: [(np.int64(0), np.int64(2))]
Ціна гри: -2.0

```

Рисунок 7.1 – Результат роботи коду

Висновок: на цій лабораторній роботі ми вивчили опанування методів спрощення й розв'язання матричних ігор. У результаті аналізу було виявлено сідлову точку, що свідчить про наявність розв'язку гри в чистих стратегіях. Оптимальна ціна гри становила $(v=-2.0)$, при цьому оптимальна стратегія для Гравця А — другий рядок вихідної матриці, а для Гравця Б — четвертий стовпець вихідної матриці. Гра є строго детермінованою, і застосування змішаних стратегій не потрібне.

Контрольні питання:

1. Конфліктна ситуація:

Ситуація, де інтереси гравців протилежні — виграш одного = програш іншого.

2. Спрощення матриці гри:

- Видаляємо стратегії, що завжди гірші (доміновані).
- Видаляємо однакові рядки або стовпці.
- Повторюємо, поки матриця не спроститься.

3. Змішані стратегії:

Гравець обирає стратегії випадково за певними ймовірностями, щоб підвищити шанс на виграш.

4. Лінійне програмування для гри:

- Гравець 1: вибирає ймовірності $x_1, \dots, x_n$, щоб мінімальний виграш був максимальним.
- Гравець 2: вибирає ймовірності $y_1, \dots, y_m$, щоб максимальний програш був мінімальним.
- Рішення дає оптимальні змішані стратегії і значення гри.